20 · Component Lifecycle Hooks

Intermediate Angular learning notes

a. Definition

Every component has a **life**: Angular creates it, updates it and finally destroys it. **Lifecycle hooks** are methods Angular calls at these moments. The most important are: `ngOnChanges` (an input changed), `ngOnInit` (ready - good place for setup), `ngAfterViewInit` (the view exists) and `ngOnDestroy` (cleanup).

Use `constructor` only for injecting dependencies. Do initial work in `ngOnInit`, and release timers, event listeners and subscriptions in `ngOnDestroy` to avoid memory leaks.

b. Diagram



Order of the most used hooks

c. Syntax

```
export class MyComponent implements OnInit, OnDestroy {
  ngOnInit()    { /* load data, start timers */ }
  ngOnDestroy() { /* unsubscribe, clear timers */ }
}
```

d. Example Program

Programs/20_Lifecycle_Hooks/lifecycle-logger.component.ts

```
import { AfterViewInit, Component, inject, Injectable, input, OnChanges, OnDestroy, OnInit,
signal, SimpleChanges } from '@angular/core';

// A tiny service so we can SEE the order of the hooks on screen
@Injectable({ providedIn: 'root' })
export class LogService {
  logs = signal<string[]>([]);
  add(message: string) {
    this.logs.update((l) => [...l, message]);
  }
}

@Component({
  selector: 'app-tea-card',
  template: `<p>Tea: {{ name() }}</p>`,
})
export class TeaCardComponent implements OnChanges, OnInit, AfterViewInit, OnDestroy {
  name = input.required<string>();
```

```
  private log = inject(LogService);

  constructor() {
    this.log.add('constructor');
  }
  ngOnChanges(changes: SimpleChanges) {
    this.log.add(`ngOnChanges -> ${changes['name'].currentValue}`);
  }
  ngOnInit() {
    this.log.add('ngOnInit');
  }
  ngAfterViewInit() {
    this.log.add('ngAfterViewInit');
  }
  ngOnDestroy() {
    this.log.add('ngOnDestroy');
  }
}

@Component({
  selector: 'app-root',
  imports: [TeaCardComponent],
  template: `
    <button id="rename" (click)="tea.set('Ginger Tea')">Rename tea</button>
    <button id="toggle" (click)="show.set(!show())">{{ show() ? 'Remove' : 'Show' }}
card</button>

    @if (show()) { <app-tea-card [name]="tea()" /> }

    <h4>Hook log</h4>
    <ol>
      @for (l of logger.logs(); track $index) { <li>{{ l }}</li> }
    </ol>
  `,
})
export class LifecycleLoggerComponent {
  logger = inject(LogService);
  show = signal(true);
  tea = signal('Masala Tea');
}
```

e. Output

Rendered in the browser (text on screen, after the UI steps below):

1. Click **Rename tea**.
2. Click **Remove card** to destroy the child.

```
Rename teaShow card
Hook log
constructor
ngOnChanges -> Masala Tea
ngOnInit
ngAfterViewInit
ngOnChanges -> Ginger Tea
ngOnDestroy
```

f. Real-World Example (English + தமிழ்)

English

A tea shop's day: open the shop (`ngOnInit`), update the price board when prices change (`ngOnChanges`), and at night switch off the stove and wash the vessels (`ngOnDestroy`).

தமிழ்

ஒரு தேநீர்க் கடையின் நாளை நினைத்துப் பாருங்கள். காலையில் கடையைத் திறப்பது (`ngOnInit`) - அடுப்பு பற்றவைத்து, பால் வாங்கி வைப்பது. நாள் முழுவதும் விலை மாறினால் பலகையைப் புதுப்பிப்பது (`ngOnChanges`).

இரவு கடையை மூடும்போது (`ngOnDestroy`) அடுப்பை அணைத்து, பாத்திரங்களைக் கழுவி, விளக்கை அணைக்க வேண்டும். இதைச் செய்யாவிட்டால் (unsubscribe, timer நிறுத்துதல்) மறுநாள் பிரச்சனை - அதுதான் memory leak.

g. Pros

- Clear points to start/stop work
- Prevents memory leaks when used correctly

h. Cons

- Many hooks to remember
- Wrong hook choice causes ExpressionChanged errors

i. Advanced / New Features

- `afterNextRender` / `afterEveryRender`
- `DestroyRef.onDestroy`
- Effects often replace some hooks

j. Common Mistakes

- Doing heavy work (like HTTP calls) in the constructor instead of `ngOnInit`.
- Forgetting to clean up intervals/subscriptions in `ngOnDestroy`.
- Accessing a child view in `ngOnInit`; it is only ready in `ngAfterViewInit`.
- Modifying state in `ngAfterViewInit` without care, leading to ExpressionChanged errors.

k. Interview Questions

Q1. Name the common lifecycle hooks in order.
constructor, ngOnChanges, ngOnInit, ngDoCheck, ngAfterContentInit, ngAfterViewInit, ngOnDestroy.

Q2. Constructor vs ngOnInit?
The constructor only creates the object and injects dependencies; `ngOnInit` runs after inputs are set and is where setup belongs.

Q3. When does ngOnChanges run?
Before `ngOnInit` and whenever an input value changes.

Q4. How do you avoid memory leaks?
Clean up in `ngOnDestroy`, use `async` pipe, `takeUntilDestroyed` or `DestroyRef.onDestroy`.

l. Practice Exercises

1. Start a `setInterval` clock in `ngOnInit` and stop it in `ngOnDestroy`.
2. Add `ngDoCheck` to the logger and observe how often it runs.
3. Use `DestroyRef` (`inject(DestroyRef).onDestroy(...)`) instead of `ngOnDestroy`.